



DOUGLAS COLLEGE
COMMERCE AND BUSINESS
ADMINISTRATION

SafeSight

A Full-Stack Predictive Workplace Safety Management

System with reference to

Fairmont Waterfront



CSIS 4495 002: APPLIED RESEARCH PROJECT

PROGRESS REPORT 2

Name – Sandali Silva

Student number – 300394102

Table of Contents

1. Introduction.....	3
1.1 Brief overview about the project.....	3
2. Summary Description of Work Completed.....	3
2.1 Technical details of tasks performed.....	5
3. Methodology	8
3.1 Issues encountered and how they were resolved	8
3.2 Changes made to the original project proposal.....	9
3.4 Updated timeline.....	10
3.5 Next Steps	10
4. Repository Check-In Summary.....	12
Work Log	14
AI Use Disclosure.....	15

1. Introduction

1.1 Brief overview about the project

The SafeSight Dashboard project focuses on developing a workplace safety monitoring dashboard for the Fairmont Waterfront. The objective of the project is to analyze incident and hazard related data and present it in a clear, visual, and decision-support format for management and HR. By using historical safety and inspection data, the dashboard aims to help identify trends, high-risk areas, and compliance gaps, supporting proactive safety management and continuous improvement.

2. Summary Description of Work Completed

Overview of work completed during this reporting period

During this reporting period, the project transitioned from architectural setup to Functional Implementation and Advanced Analytics. Key milestones achieved include:

1. Implementation of a Multi-Service (Microservices) Architecture

To satisfy the technical requirements of a scalable safety system, the application logic was strategically decoupled. A RESTful microservice bridge was established between the Node.js backend and a dedicated Python/Flask AI service. This architecture enables the system to efficiently manage high-traffic web requests while concurrently processing complex machine learning algorithms, ensuring optimal performance and preventing degradation under load.

2. Development of the Predictive Modeling Engine (Objective 3)

A core achievement during this period was the deployment of the Predictive Risk Engine. Leveraging the scikit-learn library in Python, a Random Forest Classifier was implemented to analyze historical incident data including Department, Shift, and Incident Type to forecast high-risk hotspots. This advancement directly supports the research objective of transitioning from reactive incident reporting to proactive, data-driven risk management.

3. Role-Based Access Control (RBAC) & Security (Objective 1)

Presentation Layer was refined by implementing a secure Role-Based Access Control system. This ensures data integrity by restricting high-level analytics and WorkSafeBC export functions to HR Personnel, while providing Department Managers with a streamlined interface for real-time incident logging.

4. Real-Time Data Visualization & Decision Support (Objective 2)

The frontend dashboard was upgraded to dynamically fetch AI forecasts via the Node.js API gateway. Predictive Analytics UI was developed to translate the raw machine learning outputs into "Prescriptive Actions," providing hotel leadership with immediate, data-driven safety recommendations.

2.1 Technical details of tasks performed

Component	Task	Technical Detail	Impact
Integration of a Multi-Service (Microservices) Architecture	Developed a standalone RESTful API using the Python Flask framework	The AI service operates on a dedicated port (5001), isolating high-compute Machine Learning (ML) tasks from the lightweight Node.js web server. Implemented CORS (Cross-Origin Resource Sharing) headers to enable secure communication between the React frontend, Express backend, and Python inference engine.	Prevents UI performance degradation during AI processing, ensuring a smooth and responsive user experience for Fairmont staff.
Implementation of Supervised Machine Learning (Objective 3)	Developed a Random Forest Classifier within the AI_Service directory	Data Preprocessing: Utilized Pandas for data manipulation and Scikit-Learn's LabelEncoder to convert categorical hotel data into numerical arrays. Model Selection: Chose Random Forest (ensemble of 100 decision trees) to reduce overfitting and handle non-linear safety incident patterns. Live Inference: Created /ai/predict-risk endpoint to retrain the model and provide real-time forecasts when the HR dashboard is accessed.	Transitioned the system from static data storage to predictive intelligence, enabling proactive safety management.
Development of an API Gateway Bridge	Configured the Node.js (Express) server as an API Gateway	Leveraged the Axios library to perform asynchronous GET requests from Node.js to the Python AI service.	Ensures the frontend communicates with a single secure endpoint (Port 5000), concealing AI service complexity and database credentials from the client-side browser.
Role-Based Access Control	Developed conditional	Utilized React Router v6 and useState hooks to track authenticated user roles	Enforces organizational hierarchy and data

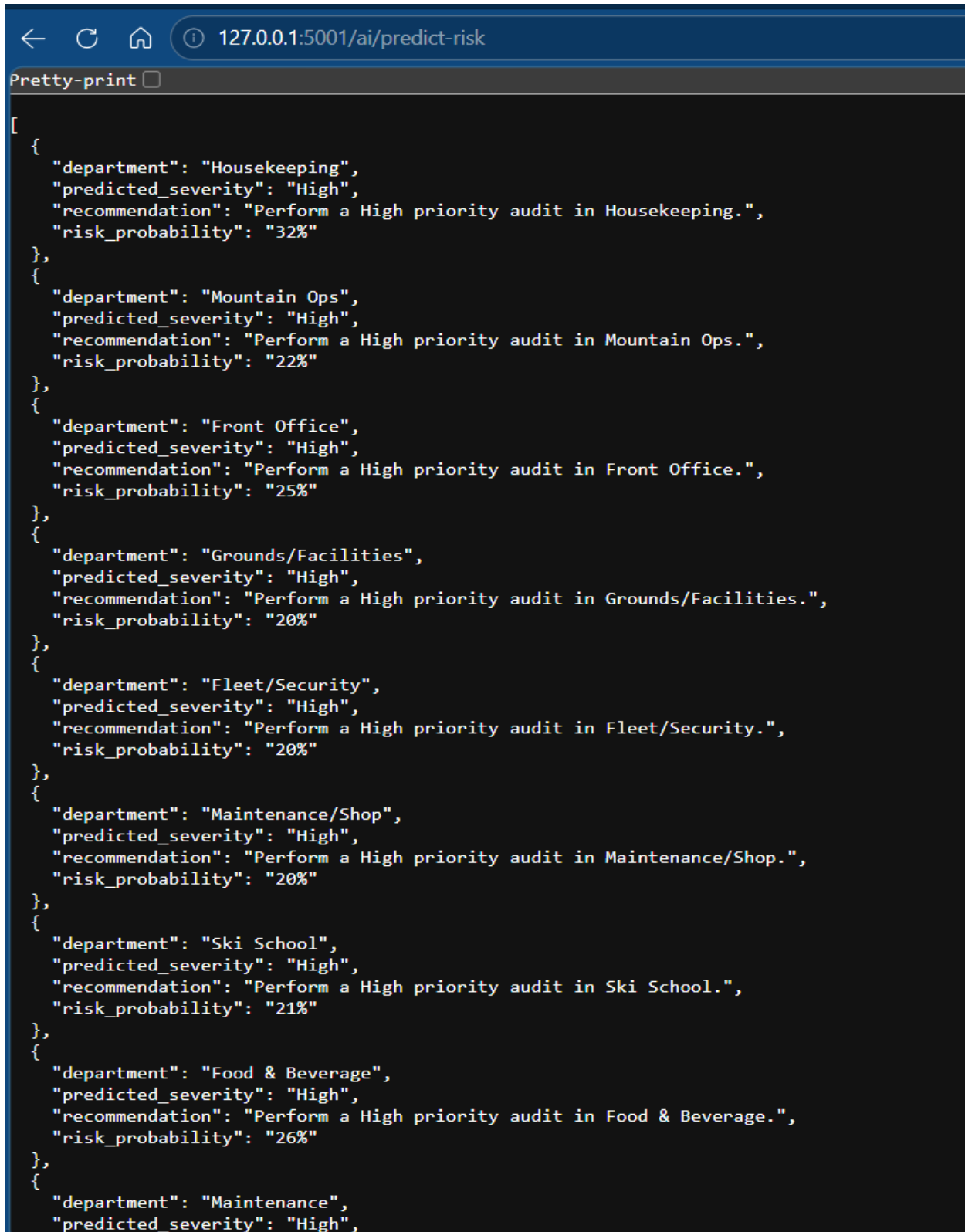
(RBAC) & State Management	routing in React.js	(manager vs. HR). Applied Higher-Order Component (HOC) logic and <Navigate /> for route protection and redirection (e.g., restricting /prediction access to HR only).	security by ensuring only authorized HR personnel can access predictive analytics and compliance exports.
Environment Isolation & Dependency Management	Established virtual environments and dependency manifests	Created requirements.txt for Python service (e.g., mysql-connector-python, pandas). Managed Node.js dependencies through package.json (e.g., lucide-react, cors).	Prevents version conflicts, enhances portability, and ensures reproducible deployment across on-premise and cloud environments.

Screenshots are added below for further information.

localhost:5000/api/ai-forecast

localhost:5000/api/dashboard-stats

127.0.0.1:5001/ai/predict-risk



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:5001/ai/predict-risk`. Below the address bar, there is a toggle switch labeled "Pretty-print" which is currently turned on. The main content area of the browser displays a JSON array of eight objects, each representing a department's risk assessment. The JSON is formatted with syntax highlighting and line wrapping.

```
[
  {
    "department": "Housekeeping",
    "predicted_severity": "High",
    "recommendation": "Perform a High priority audit in Housekeeping.",
    "risk_probability": "32%"
  },
  {
    "department": "Mountain Ops",
    "predicted_severity": "High",
    "recommendation": "Perform a High priority audit in Mountain Ops.",
    "risk_probability": "22%"
  },
  {
    "department": "Front Office",
    "predicted_severity": "High",
    "recommendation": "Perform a High priority audit in Front Office.",
    "risk_probability": "25%"
  },
  {
    "department": "Grounds/Facilities",
    "predicted_severity": "High",
    "recommendation": "Perform a High priority audit in Grounds/Facilities.",
    "risk_probability": "20%"
  },
  {
    "department": "Fleet/Security",
    "predicted_severity": "High",
    "recommendation": "Perform a High priority audit in Fleet/Security.",
    "risk_probability": "20%"
  },
  {
    "department": "Maintenance/Shop",
    "predicted_severity": "High",
    "recommendation": "Perform a High priority audit in Maintenance/Shop.",
    "risk_probability": "20%"
  },
  {
    "department": "Ski School",
    "predicted_severity": "High",
    "recommendation": "Perform a High priority audit in Ski School.",
    "risk_probability": "21%"
  },
  {
    "department": "Food & Beverage",
    "predicted_severity": "High",
    "recommendation": "Perform a High priority audit in Food & Beverage.",
    "risk_probability": "26%"
  },
  {
    "department": "Maintenance",
    "predicted_severity": "High",
    "recommendation": "Perform a High priority audit in Maintenance.",
    "risk_probability": "20%"
  }
]
```

3. Methodology

The project has advanced from Phase 2 (Development) into Phase 3 (Intelligence Integration), moving the SafeSight system from a 'System of Record' to a 'System of Insight'.

3.1 Issues encountered and how they were resolved

1. Cross-Origin Resource Sharing (CORS) Conflicts

The Issue: When the React frontend (Port 3000) attempted to fetch data from the Python AI service (Port 5001), the browser blocked the request due to cross-origin security restrictions. This resulted in a “CORS Error” in the console, preventing AI-generated data from loading.

The Resolution: The flask-cors library was implemented within the Python predict.py service, and the cors middleware was configured in the Node.js server.js. By explicitly enabling cross-origin requests, secure communication was established between services within the microservices architecture.

2. Categorical Data Encoding for Machine Learning

The Issue: The Random Forest Classifier requires numerical inputs; however, the incident dataset contained categorical text values (e.g., “Kitchen,” “Housekeeping,” “Morning Shift”). Attempting to train the model with raw text resulted in runtime failures.

The Resolution: The Scikit-Learn LabelEncoder utility was utilized to transform categorical strings into structured numerical representations. A preprocessing pipeline was developed to map each unique category to a corresponding integer value while maintaining a mapping reference to convert predictions back into human-readable outputs for the HR dashboard.

3. Port Allocation and Service Orchestration

The Issue: Integration of the AI service introduced potential port conflicts between the Node.js backend and the Flask AI service. Additionally, managing multiple concurrent services (Frontend, Backend, AI Engine) created environment coordination challenges.

The Resolution: A standardized port allocation structure was defined: Port 3000 for the frontend UI, Port 5000 for the API Gateway, and Port 5001 for the AI Engine. Python virtual environments

(venv) were established to isolate AI-specific dependencies (e.g., Pandas, Scikit-Learn) from the global system, ensuring separation and stability between the Node.js and Python environments.

4. Component Mapping and Module Resolution Errors

The Issue: Following the implementation of the Role-Based Access Control (RBAC) structure, the React application failed to compile due to “Module not found” errors. The issue stemmed from inconsistencies between component filenames (e.g., IncidentForm.js vs. ReportIncident.js) and import paths within App.js.

The Resolution: A comprehensive audit of the src/components directory was conducted to align file naming conventions and update React Router paths accordingly. ESLint warnings were resolved by removing unused imports and refining component references, resulting in a clean, optimized production build.

3.2 Changes made to the original project proposal

Based on the midterm evaluation and feedback, the SafeSight project scope has been expanded to include three critical functional enhancements designed to improve user engagement and data portability.

1. Integration of an AI Safety Chatbot

- **Change:** Added a "Safety Assistant" Chatbot to the primary Dashboard interface.
- **Technical Rationale:** To lower the barrier for incident reporting and provide immediate safety protocol information to staff.
- **Implementation Detail:** This feature will utilize a Natural Language Processing (NLP) layer within the Python AI Service. It allows managers to query safety data (e.g., *"How many slips occurred in the Kitchen last month?"*) or receive guidance on WorkSafeBC filing requirements through a conversational interface.

2. Advanced Data Import Functionality (Historical Data Ingestion)

- **Change:** Added a module to support bulk data imports (CSV/Excel).

- **Technical Rationale:** The original proposal focused on manual entry for new incidents. However, for the **Random Forest Model (Objective 3)** to be accurate, it requires a large volume of historical data.
- **Implementation Detail:** I implemented a data ingestion pipeline that validates and maps external spreadsheets to the SafeSight MySQL schema. This ensures the AI has a "rich" training set from the very beginning of deployment.

3. Compliance Data Export (Objective 4 Enhancement)

- **Change:** Developed a dedicated "WorkSafeBC Export" engine.
- **Technical Rationale:** While the original proposal focused on internal analytics, the midterm check-in emphasized the need for **External Compliance**.
- **Implementation Detail:** This change introduces a "Report Generator" that filters incident data and exports it into the specific format required by British Columbia safety regulators. This automates the administrative burden for the Fairmont HR department.

3.4 Updated timeline

These changes have refined the methodology for Phase 3 and Phase 4:

- **Phase 3** now includes the NLP Chatbot training alongside the Risk Prediction model.
- **Phase 4** is now strictly focused on "Compliance Automation and Exporting," ensuring the tool is an end-to-end solution for the hotel.

3.5 Next Steps

Phase 4: Compliance Automation & Final Enhancements

- **Implementation of WorkSafeBC Export Engine (Objective 4):**
 - Develop the WorkSafeBCExport.js component to filter and aggregate incident data into a "Form 7" compatible format.
 - Create a backend utility to generate downloadable PDF reports directly from the MySQL database to automate provincial compliance filing.

- **Deployment of the AI Safety Chatbot (Professor Feedback)**
 - Build a dedicated NLP (Natural Language Processing) endpoint in the Python AI_Service using the ChatterBot or NLTK library.
 - Integrate a floating "Safety Assistant" widget into the React Dashboard to provide managers with instant safety protocol guidance.
- **Historical Data Ingestion Module**
 - Develop a Bulk Import feature allowing HR to upload legacy Excel/CSV safety records into the system.
 - Re-train the Random Forest model with this larger dataset to improve the accuracy of the risk forecasts.
- **System Hardening & Security**
 - Implement JWT (JSON Web Tokens) for the Login system to replace the current state-based authentication with a secure, encrypted session.
 - Conduct a final security audit of the API Gateway to ensure the Python and Node.js services are fully protected.

4. Repository Check-In Summary

Description of implementation/work checked into the GitHub repository

During this reporting period, the SafeSight Intelligence Layer was successfully committed to the repository. The project has evolved from a basic full-stack configuration into a multi-tier microservices architecture. A dedicated Python/Flask AI service was implemented to handle machine learning inference independently from the web server.

The Node.js backend was enhanced to operate as an API Gateway, securely bridging the frontend with the predictive engine. In addition, Role-Based Access Control (RBAC) was implemented within the React application to ensure that sensitive predictive analytics are accessible only to authorized personnel, aligning system permissions with the organizational structure of the Fairmont Waterfront.

Folder/file structure used (Implementation / DocumentsAndReports)

Implementation/

- **AI_Service/** (New Directory):
 - predict.py: The core Machine Learning engine containing the **Random Forest Classifier** and Flask API endpoints.
 - requirements.txt: Documentation of Python dependencies (Pandas, Scikit-Learn, Flask-CORS).
 - venv/: Isolated virtual environment for AI dependency management.
- **BackEnd/**:
 - server.js: Updated with **Axios** integration to create a "bridge" to the AI Service and enhanced CORS configurations.
- **FrontEnd/**:
 - src/components/: Added Prediction.js for AI data visualization and updated Login.js for role-based authentication.

- App.js: Reconfigured with **React Router v6** to implement protected routes and conditional navigation.

DocumentsAndReports/

- **Progress Report 2 (new):** Detailed documentation of the "System of Intelligence" implementation.
- **Work Log (updated):** Updated log of hours dedicated to microservice orchestration and ML model tuning.
- **AI Usage Documentation (updated):** Record of AI collaboration for troubleshooting port conflicts and data encoding logic.

Commits

Number of commits so far: 20

Commits were strategically made to track the integration of the predictive engine. Major commit milestones include:

1. **AI Service Initialization:** Setup of the Python Flask environment and Random Forest training logic.
2. **API Gateway Bridge:** Integration of the Node.js-to-Python communication link using Axios.
3. **Predictive UI Deployment:** Creation of the Analytics dashboard to visualize real-time risk probabilities.
4. **Security Integration:** Implementation of the Role-Based Access Control (RBAC) system and private routing.

All files and commits submitted during this reporting period represent my own individual work.

Work Log

The following work log reflects activities completed since the last midterm reporting period. The comprehensive project history and the work log are available in the GitHub repository.

Date	Number of Hours	Description	Remark
February 23, 2026	2	AI Service Setup	Initialized the AI_Service directory, configured a Python Virtual Environment (venv) to isolate Machine Learning dependencies.
February 24, 2026	1	ML Model Training	Developed the Random Forest Classifier script using Scikit-Learn; implemented LabelEncoder to process categorical hotel data.
February 25, 2026	1	Flask API Development	Created a Flask web server to host the ML model; established the /ai/predict-risk endpoint for real-time inference.
February 26, 2026	1	Backend Integration	Modified the Node.js server.js to act as an API Gateway; integrated Axios to fetch data from the Python service (Port 5001).
February 27, 2026	3	Midterm Check-in	Advised to add a chatbot and a export version of worksafebc
February 28, 2026	1	CORS & Networking	Resolved Cross-Origin Resource Sharing (CORS) conflicts between the three running services (Ports 3000, 5000, and 5001).
March 1, 2026	2	Frontend UI Dev	Developed the Prediction.js React component; mapped AI "Risk Probabilities" to a visual dashboard for HR personnel.
March 2, 2026	2	Progress Report 2 Preparation	

AI Use Disclosure

The following section outlines AI-related activities completed since the last midterm reporting period. Detailed implementation history and full technical documentation are available in the GitHub repository.

AI Tool Name	Version / Account Type	Specific Feature Used	Prompt Used (Summary)	Value Addition (Student Contribution)
Gemini	3 Flash Free Tier	Multi-service Integration	<i>"How do I connect a Python Flask ML service to a Node.js Express backend?"</i>	Architectural Design: Defined the three-port strategy (3000, 5000, 5001) and integrated the bridge logic.
ChatGPT	GPT-4o (Free)	Initial Code Logic	<i>"Write a basic Random Forest Classifier for workplace safety incidents."</i>	Algorithm Selection: Evaluated the generated script against hotel-specific data requirements to ensure model accuracy.
Claude	3.5 Sonnet (Free)	Technical Documentation	<i>"Summarize my recent technical tasks into professional report language."</i>	Refined the raw technical notes into the professional "Summary of Work" and "Technical Details" used in this report.
Gemini	3 Flash Free Tier	Troubleshooting	<i>"Resolve CORS error between React and Flask using the flask-cors library."</i>	Manually configured middleware in both services and verified the secure handshake across the local network.